COMP 479/6791 Project 1 Report

Course: COMP 479/6791 — Information Retrieval and Web Search
Project: Project 1 — Reuters-21578 Indexing
Dataset: Reuters-21578
Name: Wenshuo Pei
Student ID: 40281918
Date: 2026/10/18

1. Introduction

The purpose of this project is to implement and compare two inverted-index construction methods on Reuters-21578: a naive indexer and a SPIMI-inspired indexer. The project also includes single-term and AND query processing, a lossy preprocessing experiment, and a performance comparison between the two indexing approaches.

Reuters-21578 [2] is stored as a set of SGML files. The `NEWID` value of each Reuters article is used as its document identifier so that all indexers and query processors work with the same document IDs.

The project is divided into four main parts:

1. a naive indexer that first creates term-document pairs;
2. single-term and AND query processing;
3. lossy preprocessing and compression experiments;
4. a SPIMI-inspired indexer that builds postings directly.

2. Data Preprocessing

The Reuters collection contains 22 SGML files and 21,578 documents. Each article is extracted from a `<REUTERS> ... </REUTERS>` block. The `NEWID` attribute is converted to an integer and used as the DocID.

For the normal uncompressed index, the text from the `TITLE` and `BODY` fields is extracted and tokenized. Tokens are converted to lowercase, and tokens that contain no alphanumeric characters are removed.

The preprocessing module produces documents in the following form:

```
(docID, [token1, token2, token3, ...])
```

This common document stream is reused by both the naive indexer and the SPIMI-inspired indexer.

3. Subproject I — Naive Indexer

3.1 Method

The naive indexer follows the required term-document pair approach.

For every token in every document, the indexer first creates a pair:

```
(term, docID)
```

All pairs are stored in a list `F`. After all documents have been processed, the list is sorted and duplicate pairs are removed. The sorted pairs are then converted into an inverted index of the form:

```
term -> [docID1, docID2, ...]
```

For example, if a term occurs in documents 1, 5, and 20, its final index entry is:

```
term -> [1, 5, 20]
```

3.2 Implementation

The naive implementation is divided into three main steps:

- `build_pair_list()` creates the list of term-document pairs;
- `sort_and_deduplicate()` sorts the list and removes repeated pairs;
- `build_inverted_index()` groups DocIDs into postings lists.

The complete Reuters-21578 index contains **74,870 distinct terms** using the normal project preprocessing pipeline.

The main disadvantage of this method is the intermediate list `F`. For a large collection, the complete set of term-document pairs must be stored and globally sorted before the final postings lists can be built.

4. Subproject II — Query Processing

4.1 Single-term queries

A single-term query performs a dictionary lookup and returns the postings list for the term. If the term is not present, an empty list is returned.

Three sample single-term queries were used:

Query	Number of matching documents
`cocoa`	97
`bahia`	15
`review`	373

4.2 AND queries

AND queries are evaluated using postings-list intersection. Because postings lists are sorted by DocID, two pointers can move through both lists without comparing every possible pair of entries.

Three sample AND queries were used:

Query	Number of matching documents
`cocoa AND bahia`	10
`cocoa AND review`	5
`bahia AND review`	4

The result for `cocoa AND bahia` is:

```
[1, 11459, 13462, 13487, 13650, 14343, 14372, 14511, 14651, 17568]
```

These tests confirm that the query processor returns the intersection of the postings lists rather than scanning the original documents again.

5. Subproject III — Compression Experiment

A separate compression experiment was performed using the preprocessing stages from Table 5.1 of the course textbook [1].

Numbers are tokens that parse with Python `float()` after commas and a trailing `%` are removed. The 30 and 150 stopwords are the most frequent terms of the case-folded Reuters-21578 collection, not a standard stopword list, and both are removed from the case-folded collection. Stemming uses NLTK `PorterStemmer` and is applied to the 150-stopword collection. The exact definitions are given in the Compression Experiment document.

The measured values were:

Stage	Terms	Term Cml	Postings	Posting Cml
Unfiltered	91,280	—	1,723,515	—
No numbers	69,375	-24.0%	1,611,025	-6.5%
Case folding	52,967	-42.0%	1,493,162	-13.4%
30 stopwords	52,937	-42.0%	1,214,289	-29.5%
150 stopwords	52,817	-42.1%	933,340	-45.8%
Stemming	42,008	-54.0%	889,510	-48.4%

The detailed comparison with Table 5.1 is provided in the separate **Compression Experiment** document.

5.1 Effect on retrieval results

The six sample queries from Subproject II were also executed on the compressed index.

Query	Original index	Compressed index
`cocoa`	97	97
`bahia`	15	15
`review`	373	488
`cocoa AND bahia`	10	10
`cocoa AND review`	5	8
`bahia AND review`	4	4

The results for `cocoa` and `bahia` do not change, which means that the preprocessing does not change these terms in a way that affects retrieval.

The result count for `review` increases from 373 to 488, and `cocoa AND review` increases from 5 to 8. The main reason is stemming. Different morphological forms related to `review` can be mapped to the same stem, so the compressed index can return documents that would not match the exact surface form `review` in the original index.

This increases the number of matching documents and can improve recall, but a larger result set does not automatically mean that retrieval precision or overall accuracy has improved.

6. Subproject IV — SPIMI-Inspired Indexer

6.1 Method

The SPIMI-inspired indexer reuses the same document stream as the naive indexer, but it does not create the complete intermediate list `F`.

Instead, each term is inserted directly into a dictionary and the current DocID is appended to the term's postings list. Duplicate occurrences of a term inside the same document are removed before insertion.

Conceptually, the difference is:

```
Naive:
term-docID pairs -> complete list F -> global sort -> postings lists

SPIMI-inspired:
term -> directly append DocID to postings list
```

This avoids storing and globally sorting the full term-document pair list.

6.2 Correctness

The complete SPIMI-inspired index also contains **74,870 distinct terms**. The implementation was checked against the naive index, and the two indexes were identical for the same input documents.

This confirms that the two methods produce the same uncompressed inverted index even though they build it differently.

6.3 Timing comparison

The assignment requires a timing comparison for **10,000 term-document pairings**. To reduce timing noise, both indexers were executed 20 times on the same 10,000-token input and the average time was calculated.

The measured results were:

Indexer	Average time
Naive	0.005808 s
SPIMI-inspired	0.001528 s

The SPIMI-inspired implementation was approximately **3.80 times faster** in this experiment.

The result is consistent with the structure of the algorithms. The naive method first materializes a complete pair list and then sorts it, while the SPIMI-inspired method uses hash-table lookup and direct postings insertion. For a small input, both methods finish very quickly, but the additional sorting work of the naive method becomes more important as the amount of input grows.

The measured speedup should be interpreted as an experimental result on this machine and implementation, not as a universal constant.

7. Testing and Validation

Automated tests were written for the preprocessing, naive indexing, query processing, and SPIMI-inspired indexing modules.

The test suite checks examples including:

- Reuters article splitting and `NEWID` extraction;
- title/body text extraction and tokenization;
- term-document pair construction;
- sorting and duplicate removal;
- inverted-index construction;
- single-term lookup;
- postings intersection;
- AND queries with no result;
- SPIMI-inspired index construction;
- equality between the naive and SPIMI-inspired indexes.

The complete test suite contains **15 tests**, and all **15 tests passed**.

In addition to unit tests, the index was checked on the full Reuters-21578 collection using known query outputs.

8. Discussion

The project illustrates that the same inverted index can be produced using different construction strategies.

The naive method is simple and closely follows the theoretical construction process. Its main cost is the large intermediate term-document pair list and the global sorting step.

The SPIMI-inspired method builds postings directly and therefore avoids this intermediate structure. This is why it is more suitable as the collection size grows. On very small data, the wall-clock difference can be difficult to notice because both implementations finish quickly, but the difference in algorithm structure is still important for scalability.

The compression experiment shows a different type of trade-off. Removing numbers, folding case, removing stopwords, and stemming reduce index size in different ways. Stopword removal mainly reduces postings, while stemming mainly reduces the number of dictionary terms. These transformations can also change retrieval behavior, as shown by the increased result count for `review` after stemming.

Together, the experiments show that indexing efficiency, storage requirements, and retrieval behavior are all influenced by preprocessing and data-structure choices.

9. Challenge Queries

The official 24-hour challenge queries were:

1. `Carmark`
2. `copper`
3. `Chrysler and Bundesbank`

All three were executed on both the naive index and the SPIMI-inspired index with:

```
.\.venv\Scripts\python.exe -m scripts.run_challenge
```

The script writes its output to `results/challenge/challenge_results.txt` and `results/challenge/challenge_timings.txt`. All values in this section are copied from these two files.

9.1 Setup

The Reuters-21578 collection is parsed and tokenized once with `iter_documents()`, and the same list of `(docID, tokens)` documents is passed to both `build_naive_index()` and `build_spimi_index()`. Both indexes therefore use exactly the same preprocessing as the rest of the project: title and body text, NLTK tokenization, lowercasing, and removal of tokens without letters or digits.

Queries reuse the existing query processor. A one-word query is answered with `single_term_query()`, and a multi-word query with `and_query()`. Query words are lowercased by `single_term_query()`, so `Carmark` and `carmark` are the same query.

In the third query, `and` is interpreted as the Boolean AND operator, not as a search term. The query string is split into words and `and` is removed before the remaining words are passed to `and_query()`:

Query	Operands
`Carmark`	`['Carmark']`
`copper`	`['copper']`
`Chrysler and Bundesbank`	`['Chrysler', 'Bundesbank']`

This matters because the uncompressed index contains the term `and`. Passing it as a third operand would make the query require the word "and" in every returned document.

9.2 Results

Query	Naive index	SPIMI-inspired index	Identical
`Carmark`	1	1	True
`copper`	117	117	True
`Chrysler AND Bundesbank`	0	0	True

Matching DocIDs:

```
Carmark:
[19758]

copper:
[22, 793, 800, 816, 922, 1148, 1184, 1552, 1607, 2006, 2074, 2186, 2764, 2782, 2880, 3454,
3613, 3862, 4291, 4373, 4431, 4744, 4816, 5203, 5209, 5435, 5788, 5827, 5888, 6025, 6225,
6395, 6846, 6927, 7552, 7724, 7775, 8569, 8601, 11053, 11273, 11945, 12024, 12215, 12223,
12243, 12484, 12489, 12513, 12633, 12641, 12651, 12683, 12692, 12725, 12857, 12861, 12910,
12926, 12963, 12980, 12992, 13003, 13465, 13495, 13608, 13665, 13670, 13675, 13687, 13694,
13757, 13877, 13897, 14136, 14274, 14297, 14302, 14398, 14476, 14499, 14572, 14687, 14805,
14852, 15166, 15264, 15556, 15831, 15932, 15988, 16110, 16123, 16971, 17007, 17118, 17714,
18225, 18280, 18317, 18339, 18392, 18394, 18430, 18457, 18631, 18691, 18758, 18849, 18952,
19502, 19781, 19786, 19878, 20096, 21293, 21327]

Chrysler AND Bundesbank:
[]
```

`Carmark` occurs in only one article, NEWID 19758, which reports Baxter's agreement to acquire Carmark.

The empty result for `Chrysler AND Bundesbank` is a real empty intersection, not a lookup failure. Both terms are in the index: 125 documents contain `chrysler` and 167 documents contain `bundesbank`, but no document contains both. This was checked separately on the same preprocessed documents (see the demo).

9.3 Validation

The following checks were performed by the challenge script for every query:

- the naive and SPIMI-inspired results were compared as exact DocID lists;
- every result list was checked to be strictly increasing, which means it is sorted and has no duplicate DocIDs;
- an independent scan of all preprocessed documents, which does not use either index, was used to compute the expected DocIDs, and both index results were compared with it.

In addition, the two complete index dictionaries were compared. All checks passed:

Check	Result
Naive and SPIMI results identical (3 queries)	True
DocIDs sorted and unique (both indexes)	True
Both indexes match the independent document scan	True
Complete naive and SPIMI indexes identical	True (74,870 terms each)

The script exits with a non-zero status if any of these checks fails. The final run exited with status 0.

The independent scan uses the same tokenization as the indexes, so it verifies index construction and query processing, not the tokenization rules themselves.

9.4 Timing

Three kinds of time were measured separately with `time.perf_counter()` in a single run on the full collection of 21,578 documents:

Step	Time
Corpus parsing and tokenization (shared by both indexes)	12.477 s
Naive index construction	2.179 s
SPIMI-inspired index construction	0.313 s

For the full collection, the SPIMI-inspired construction was about 7.0 times faster than the naive construction in this run. This is consistent with the 10,000-pair benchmark in Section 6.3, where SPIMI was 3.80 times faster, and the difference is larger on the full collection because the naive method has to build and sort a much larger list `F`. Most of the total running time is spent parsing and tokenizing the SGML files, not building the index.

A single query takes well under a microsecond, so one measurement would mostly be timer noise. Each query was therefore run 10,000 times on each already-built index, and the average time per query is reported. Index construction, the validation scan, and file output are not included in these times.

Query	Naive index	SPIMI-inspired index
`Carmark`	0.094 µs	0.092 µs
`copper`	0.093 µs	0.090 µs
`Chrysler AND Bundesbank`	14.905 µs	15.020 µs

Query times are practically the same for both indexes. This is expected, because both methods produce the same data structure, a Python dictionary from terms to sorted DocID lists, and the same query functions are used. A single-term query is one dictionary lookup. The AND query is slower because it walks through two postings lists of 125 and 167 DocIDs. The small differences between the two indexes are within measurement noise, so the measurements do not show that either index answers queries faster. The advantage of the SPIMI-inspired method is in index construction, not in query processing.

All times depend on the machine and change slightly between runs.

9.5 Observations

- Both construction methods produced the same index, so they return the same answers for every query; they differ only in how the index is built.
- The postings lists produced by the SPIMI-inspired method are sorted because Reuters-21578 is read in increasing NEWID order. The challenge script checks this explicitly instead of assuming it.
- Only the `TITLE` and `BODY` text is indexed. Words that appear only in other fields, such as `TOPICS` or the `BC-CHRYSLER` slug lines, are not counted.
- The NLTK tokenizer does not split hyphenated words, so forms such as `silver-copper` or `Renault-Chrysler` are separate terms and are not counted as `copper` or `chrysler`. The naive index, the SPIMI-inspired index, and the validation scan all handle these words in the same way.
- The indexes are built in memory each time the script runs; they are not saved to disk, so no index loading time is reported.

10. Conclusion

This project implemented two inverted-index construction methods for Reuters-21578 and verified that both methods produce the same retrieval structure.

The naive indexer builds a complete term-document pair list, sorts it, removes duplicates, and then constructs postings. The SPIMI-inspired indexer directly builds the dictionary and postings while reading documents. On the 10,000-pair benchmark, the SPIMI-inspired implementation was faster in this experiment.

The query processor successfully supports both single-term and AND queries using dictionary lookup and postings intersection. The compression experiment also shows that lossy preprocessing can substantially reduce index size, although it can change the set of documents returned by a query.

Overall, the project demonstrates the relationship between preprocessing, inverted-index construction, query processing, index size, and indexing performance.

11. Generative AI Use Disclosure

Generative AI was used as a support tool during this project to explain information-retrieval concepts, review and discuss implementation ideas, help diagnose and refine parts of the code. The implementation was run, tested, debugged, and verified against the Reuters-21578 data by the author.

References

[1] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge University Press, 2008. Chapter 5, Table 5.1.

[2] D. D. Lewis, *Reuters-21578 Text Categorization Test Collection*, Distribution 1.0. http://www.daviddlewis.com/resources/testcollections/reuters21578/